AUDITORIA DE SEGURANÇA DE APLICAÇÃO

Relatório de Auditoria de Segurança — G-Windows

Data: 18 de outubro de 2026

Escopo auditado

Todo o código-fonte do repositório: GWindows.Core (biblioteca de modelo/regras) e GWindows.App (aplicativo WPF), mais os testes, arquivos de projeto, manifesto e nuget.config — 24 arquivos, 1951 linhas, 18 deles em C#. Binários gerados (bin/obj) e o ambiente virtual desta auditoria ficaram fora do escopo.

Nota metodológica

A stack detectada é um **aplicativo desktop Windows local** (C#, .NET 8, WPF/WinForms), sem servidor, sem banco de dados, sem autenticação e sem requisições de rede. Por isso a checklist de 22 categorias, pensada para aplicações web, foi mapeada assim: categorias de servidor/web (SQL Injection, XSS, CSRF, CORS, cookies, tokens, rate limit, autenticação, IDOR, enumeração, rotas administrativas, prompt injection, senhas, força bruta) foram marcadas **não se aplica**, com a justificativa verificada no código. As categorias com equivalente local foram avaliadas por analogia: o *layout.json* e o arquivo de importação equivalem a entrada não confiável (validação, upload); caminhos UNC em itens de cerca equivalem a SSRF local (o sistema contacta um host controlado por terceiros); leitura de arquivos grandes e regex equivalem a abuso de recursos (DoS).

Como foi verificado: leitura integral de todos os arquivos com numeração de linhas; buscas por padrões (segredos, rede, execução de processos, desserialização); *dotnet list package --vulnerable --include-transitive*; e programas de comprovação executados num projeto isolado fora do repositório, usando o carregador e as regras reais (nulos aceitos, geometria extrema, tempo do regex, consumo de memória). **Limites:** o comportamento SMB/NTLM do Windows e a partida completa do aplicativo com um layout hostil não foram exercitados; os achados A-01, A-02 e A-06 indicam o que foi confirmado e o que decorre da leitura do código e do comportamento documentado do sistema.

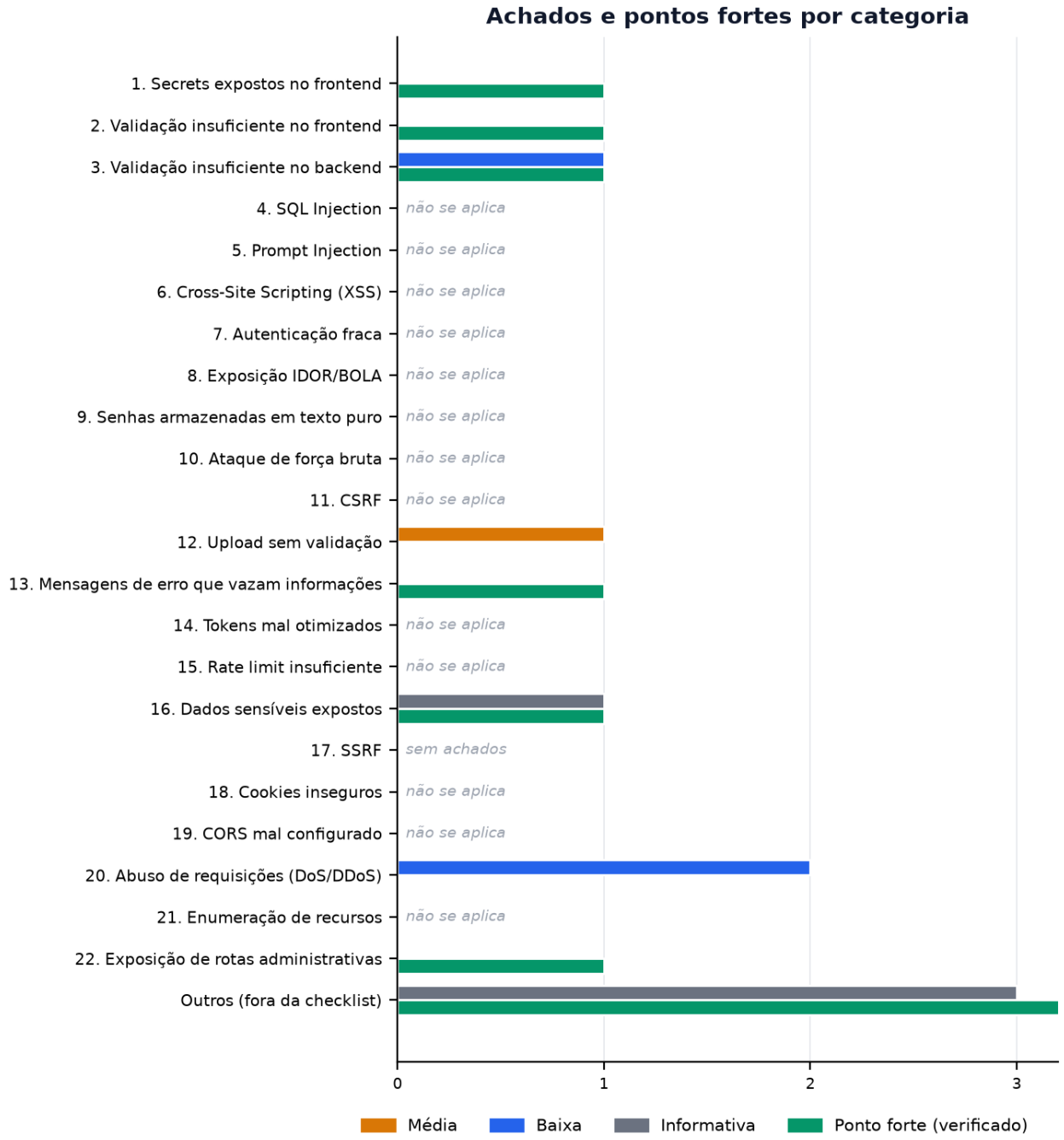
Resumo executivo

Não foram encontradas falhas críticas nem altas. Foram verificados **8 achados**: 1 de severidade média, 3 baixas e 4 informativos. O risco central é tratar o arquivo de layout (importado, restaurado ou adulterado em disco) como confiável: ele define quais caminhos o aplicativo acessa, enumera, observa e abre com um clique, e não passa por nenhuma validação. As demais falhas são negações de serviço locais de baixo impacto. Em contrapartida, 11 pontos fortes foram comprovados, entre eles a ausência de rede, de segredos e de elevação de privilégio. **Depois da auditoria, os achados A-02, A-03, A-04 e A-06 foram corrigidos, A-01 e A-07 foram mitigados e A-05 e A-08 foram aceitos** (status por achado na seção de detalhes; os trechos e linhas citados referem-se ao código no momento da auditoria).

Severidade	Quantidade	Achados
CRÍTICA	**0**	—
ALTA	**0**	—
MÉDIA	**1**	A-01
BAIXA	**3**	A-02, A-03, A-04
INFORMATIVA	**4**	A-05, A-06, A-07, A-08
PONTO FORTE	**11**	verificados com evidência (ver seção seguinte)



Achados e pontos fortes por categoria



Cada categoria é contada uma vez pela sua categoria principal (o achado A-01 aparece em 12; o cruzamento com SSRF está descrito no texto). 'Não se aplica' indica categoria sem equivalente na stack.

Pontos fortes e pontos fracos

Pontos fortes (verificados)

Categoria	O que está protegido	Evidência
1. Secrets expostos no frontend	**Nenhum segredo, credencial ou chave no código.** Busca por password|secret|token|api key e URLs em src/ (*.cs, *.xaml, *.csproj): sem ocorrências; única URL: https://api.nuget.org em nuget.config.	src/**, nuget.config
16. Dados sensíveis expostos	**Sem rede e sem telemetria.** Nenhum uso de HttpClient, Socket, WebBrowser ou WebView no código; o aplicativo só lê/escreve arquivos locais e o registro do usuário.	src/** (busca por Http*, Web*)
2. Validação insuficiente no frontend	**Colar caminhos valida existência.** Cada caminho colado só é adicionado se Directory.Exists/File.Exists for verdadeiro, com remoção de aspas e barra final.	FenceWindow.cs:236-241
3. Validação insuficiente no backend	**Desserialização segura de JSON.** System.Text.Json em tipos concretos (LayoutFile/FenceData), sem polimorfismo, sem BinaryFormatter, XmlSerializer ou carregamento dinâmico de assemblies.	Layout.cs:49; App.xaml.cs:227
13. Mensagens de erro que vazam informações	**Erros não vazam informações.** Falhas ao abrir itens/ícones são capturadas e descartadas; nenhuma mensagem com caminho, pilha ou detalhe interno é mostrada ao usuário.	FenceWindow.cs:255, 380, 398; Palette.cs:54
22. Exposição de rotas administrativas	**Executa sem privilégios elevados.** app.manifest não declara requestedExecutionLevel (padrão asInvoker); nenhuma operação exige administrador.	app.manifest:1-14
Outros (fora da checklist)	**Gancho de mouse mínimo.** O WH_MOUSE_LL só trata o clique esquerdo (0x201), lê posição e classe da janela sob o cursor e repassa com CallNextHookEx; não registra teclas nem conteúdo de outras janelas. O atalho global usa RegisterHotKey.	Native.cs:53-77, 84-91
Outros (fora da checklist)	**O aplicativo não apaga nem move arquivos do usuário.** Únicas remoções/movimentações: snapshots dentro da própria pasta (limite de 50/30 dias) e o arquivo temporário da gravação atômica. G-Windows guardam apenas referências.	App.xaml.cs:192; Layout.cs:62
Outros (fora da checklist)	**Gravação atômica com backup.** Escrita em arquivo temporário, File.Replace com .bak; evita corrupção por interrupção.	Layout.cs:55-63
Outros (fora da checklist)	**Dependências sem vulnerabilidades conhecidas.** dotnet list package --vulnerable --include-transitive em 18/10/2026: nenhum pacote vulnerável nos três projetos (único pacote de runtime: System.Drawing.Common 8.0.10).	GWindows.App.csproj:15
Outros (fora da checklist)	**Chave Run entre aspas.** O caminho gravado em HKCU\...\Run é envolvido por aspas, evitando sequestro por caminho com espaço sem aspas.	App.xaml.cs:23

Pontos fracos (riscos centrais)

Risco	Descrição
Layout tratado como confiável (A-01, A-02)	Entrada externa (importação, snapshot adulterado, layout.json editado) é aceita sem validar caminhos, nulos ou faixas: risco de acesso a recursos de rede, execução com um clique e queda na abertura.
Operações sem limites de recurso (A-03, A-04)	Regex sem timeout e leitura de linha inteira executadas na thread de interface permitem congelar o aplicativo ou consumir gigabytes de memória.
Dados em claro e endurecimento local (A-05 a A-08)	Notas copiadas em claro para snapshots/exportações; execução do Explorer sem caminho completo; mutex previsível; binário não assinado em pasta gravável.

Cobertura da checklist (22 categorias)

Resultado por categoria: **Achado** (há falha verificada), **Parcial** (parte protegida, parte com achado), **Correto** (verificado e protegido) ou **N/A** (sem equivalente na stack, com justificativa).

#	Categoria	Resultado	Justificativa / evidência
1	**Secrets expostos no frontend**	CORRETO	Busca por password|secret|token|api key|http(s)|Http* em src/ (*.cs, *.xaml, *.csproj) não retorna segredo algum; a única URL do repositório é o feed nuget.org em nuget.config (build, não runtime).
2	**Validação insuficiente no frontend**	PARCIAL	Colar caminhos valida existência antes de aceitar (FenceWindow.cs:238-241). Já a regra de captura e o caminho de portal digitados em Prompt não são validados (ver A-03 e A-01).
3	**Validação insuficiente no backend**	BAIXA	Não há backend. O equivalente é o layout.json/importação, entrada não confiável aceita sem validação de schema, nulos ou faixas (A-02).
4	**SQL Injection**	N/A	Sem banco de dados nem consultas SQL (busca por 'Sql' sem ocorrências).
5	**Prompt Injection**	N/A	Sem integração com LLM; nenhum texto de usuário é enviado a modelo.
6	**Cross-Site Scripting (XSS)**	N/A	Aplicação WPF sem WebBrowser/WebView; todo texto é renderizado por TextBlock.Text (sem interpretação de HTML/markup).
7	**Autenticação fraca**	N/A	Sem autenticação: aplicativo local de usuário único.
8	**Exposição IDOR/BOLA**	N/A	Sem recursos de múltiplos usuários nem identificadores de objeto expostos.
9	**Senhas armazenadas em texto puro**	N/A	O aplicativo não armazena senhas nem credenciais.
10	**Ataque de força bruta**	N/A	Sem formulário de login ou endpoint a ser atacado.
11	**CSRF**	N/A	Sem requisições HTTP nem sessões de navegador.
12	**Upload sem validação**	MÉDIA	Equivalente: importação de layout e restauração de snapshot (App.xaml.cs:213, 223) aceitam arquivo JSON externo sem validação (A-01).
13	**Mensagens de erro que vazam informações**	CORRETO	Nenhuma mensagem de erro com caminho, pilha ou dado interno é exibida: exceções são capturadas e descartadas (ex.: FenceWindow.cs:255, Palette.cs:54).
14	**Tokens mal otimizados**	N/A	Sem tokens de sessão/API.
15	**Rate limit insuficiente**	N/A	Sem endpoints; nada a limitar por taxa.
16	**Dados sensíveis expostos**	INFORMATIVA	O widget Notas grava texto livre em claro em layout.json, em snapshots e na exportação (A-05). Sem telemetria nem envio de dados.
17	**SSRF**	MÉDIA	Sem requisições HTTP. O análogo local é o acesso automático a caminhos UNC vindos de layout importado (A-01).
18	**Cookies inseguros**	N/A	Sem cookies.
19	**CORS mal configurado**	N/A	Sem servidor HTTP/CORS.
20	**Abuso de requisições (DoS/DDoS)**	BAIXA	DoS local: regex sem limite de tempo na regra de captura (A-03) e leitura de linha inteira na pré-visualização (A-04).
21	**Enumeração de recursos**	N/A	Sem recursos endereçáveis por rede.
22	**Exposição de rotas administrativas**	N/A	Sem rotas. O processo roda sem elevação (app.manifest não pede requireAdministrator).

Achados detalhados por categoria

3. Validação insuficiente no backend

Severidade	Arquivo:linha	Descrição
BAIXA	Layout.cs:49-50 App.xaml.cs:93-95 FenceWindow.cs:267, 271, 279 Rules.cs:21	**A-02** — layout.json sem validação de schema, nulos ou faixas: falha na inicialização e valores extremos aceitos *Status: Corrigido*

BAIXA **A-02** — layout.json sem validação de schema, nulos ou faixas: falha na inicialização e valores extremos aceitos

Status após a revisão de 18 de outubro de 2026: Corrigido. LayoutStore.Normalize remove nulos e limita tamanhos/contagens/faixas; qualquer falha de leitura cai para o .bak e depois para o padrão; arquivo acima de 20 MB é ignorado (Layout.cs). Teste automatizado cobre o caso.

Onde (arquivo:linha) e trecho — estado auditado

src/GWindows.Core/Layout.cs:49-50

```
try { if (File.Exists(p)) return JsonSerializer.Deserialize<LayoutFile>(File.ReadAllText(p)) ?? new(); }
catch (JsonException) { }
```

src/GWindows.App/App.xaml.cs:93-95

```
if (Layout.Fences.Count == 0) ...; foreach (var f in Layout.Fences) Fit(f); foreach (var f in Layout.Fences.ToList())
Open(f);
```

src/GWindows.App/FenceWindow.cs:267, 271, 279

```
bool Manual => _d.Portal.Length == 0 && _d.Smart.Length == 0 && _d.Kind.Length == 0; // e _d.Items.Where(...) /
_d.Portal.Length
```

src/GWindows.Core/Rules.cs:21

```
fences.Any(f => f.Items.Contains(path)) ? null : fences.FirstOrDefault(f => f.Rule.Length > 0 && Matches(f.Rule,
path));
```

Por que é explorável

Um JSON sintaticamente válido com "Fences": null, "Items": null ou "Name": null é aceito por LayoutStore.Load (teste executado: Fences is null: True; Items/Name/Excluded null: True) e depois desreferenciado em vários pontos (teste: Rules.Target com Items=null lança NullReferenceException). O fallback para layout.json.bak só cobre JsonException (Layout.cs:50), logo não protege nesse caso. Valores como W=2147483647, H=-5, X=2147483647 também são aceitos sem restrição (teste).

Condições de explorabilidade e verificação

Exige escrever em %AppData%\G-Windows\layout.json (mesmo usuário) ou importar um arquivo adulterado. Na importação a falha ocorre em Reload antes de persistir, então não vira laço de falha; com o arquivo principal adulterado a falha se repete a cada inicialização até o arquivo ser removido. A falha de inicialização foi demonstrada no nível de dados e de consumo (Rules.Target); a partida completa do aplicativo com arquivo hostil não foi executada para não derrubar a sessão do usuário.

Impacto

Negação de serviço local do aplicativo (queda na abertura) e janelas com geometria absurda.

Correção sugerida

Após desserializar, normalizar: substituir nulos por valores padrão, limitar W/H/X/Y a faixas razoáveis, limitar quantidade de cercas/itens e tamanho dos textos; capturar qualquer Exception (não só JsonException) e cair para .bak/padrão.

12. Upload sem validação

Severidade	Arquivo:linha	Descrição
MÉDIA	App.xaml.cs:213 App.xaml.cs:223 Layout.cs:49 FenceWindow.cs:271, 279 FenceWindow.cs:395 FenceWindow.cs:255, 351 Palette.cs:54	**A-01** — Layout importado ou restaurado não é validado: caminhos UNC/remotos são acessados e executados com 1 clique *(relacionado a: 17. SSRF)* *Status: Mitigado*

> MÉDIA **A-01** — Layout importado ou restaurado não é validado: caminhos UNC/remotos são acessados e executados com 1 clique

Status após a revisão de 18 de outubro de 2026: Mitigado. Caminhos de rede continuam permitidos por decisão do usuário. O layout agora é normalizado ao carregar e a importação/restauração pede confirmação quando o arquivo traz caminhos de rede (App.xaml.cs, ImportFrom).

Onde (arquivo:linha) e trecho — estado auditado

src/GWindows.App/App.xaml.cs:213

```
snaps.DropDownItems.Add(f.Name[..^5], null, (_, _) => { Snapshot(); Reload(LayoutStore.Load(f.FullName).Fences); });
```

src/GWindows.App/App.xaml.cs:223

```
if (d.ShowDialog() == System.Windows.Forms.DialogResult.OK) { Snapshot(); Reload(LayoutStore.Load(d.FileName).Fences);
}
```

src/GWindows.Core/Layout.cs:49

```
try { if (File.Exists(p)) return JsonSerializer.Deserialize<LayoutFile>(File.ReadAllText(p)) ?? new(); }
```

src/GWindows.App/FenceWindow.cs:271, 279

```
return _d.Items.Where(p => File.Exists(p) || Directory.Exists(p)); // e
Directory.EnumerateFileSystemEntries(_d.Portal) (l.271)
```

src/GWindows.App/FenceWindow.cs:395

```
using var ic = System.Drawing.Icon.ExtractAssociatedIcon(path);
```

src/GWindows.App/FenceWindow.cs:255, 351

```
static void Launch(string p) { ... Process.Start(new ProcessStartInfo(p) { UseShellExecute = true }) ... } // if
(click) Launch(path);
```

src/GWindows.App/Palette.cs:54

```
() => { try { Process.Start(new ProcessStartInfo(path) { UseShellExecute = true }); } catch (Exception) { } }
```

Por que é explorável

O JSON importado define Items e Portal livremente. Ao abrir cada cerca, o aplicativo consulta File.Exists/Directory.Exists, enumera o portal, observa a pasta com FileSystemWatcher e extrai o ícone do arquivo; se o caminho for \\servidor\share\x.exe, o Windows contacta o servidor SMB do atacante (tentativa de autenticação NTLM, com vazamento do hash). O rótulo vem do nome do arquivo e o ícone do próprio executável remoto, então o atacante controla a aparência (p.ex. um falso 'Chrome'). Como o item agora abre com um clique, um clique executa o programa remoto.

Condições de explorabilidade e verificação

Exige que o usuário importe um .json de origem não confiável (ou restaure um snapshot adulterado) e depois clique no item. O Windows pode exibir aviso de segurança para executáveis de rede; o vazamento de NTLM ocorre antes de qualquer clique, no carregamento. Confirmado por leitura do código e por teste com o carregador real: o arquivo com Portal e Items UNC foi aceito sem alteração (saída do teste: Portal=\\attacker.example\share). O comportamento SMB do Windows não foi exercitado (sem servidor de teste).

Impacto

Vazamento de credenciais NTLM, execução de código remoto com interação do usuário e aparência de atalho falsificada.

Correção sugerida

Validar o layout importado: aceitar só caminhos totalmente qualificados em unidades locais, rejeitar UNC (\\) e prefixos de dispositivo (\\?\, \\.\); limitar tamanho do arquivo; exigir confirmação mostrando quantos itens/caminhos externos o arquivo traz; não resolver (File.Exists) caminhos de rede sem consentimento.

16. Dados sensíveis expostos

Severidade	Arquivo:linha	Descrição
INFORMATIVA	FenceWindow.cs:296-297 Layout.cs:24 App.xaml.cs:191, 218	**A-05** — Texto do widget Notas é gravado em claro e replicado em snapshots e exportações *Status: Aceito*

> INFORMATIVA **A-05** — Texto do widget Notas é gravado em claro e replicado em snapshots e exportações

Status após a revisão de 18 de outubro de 2026: Aceito. Sem alteração: depende de o usuário não guardar dados sensíveis nas Notas.

Onde (arquivo:linha) e trecho — estado auditado

src/GWindows.App/FenceWindow.cs:296-297

```
var box = new TextBox { Text = _d.Notes, ... };
box.TextChanged += (_, _) => { _d.Notes = box.Text; App.SaveSoon(); };
```

src/GWindows.Core/Layout.cs:24

```
public string Notes { get; set; } = "";
```

src/GWindows.App/App.xaml.cs:191, 218

```
File.Copy(LayoutPath, Path.Combine(SnapDir, "...".json"), true); // File.Copy(LayoutPath, d.FileName, true);
```

Por que é explorável

Qualquer texto digitado nas notas (inclusive senhas ou dados pessoais colados) permanece em layout.json e é copiado para até 50 snapshots (30 dias) e para o arquivo exportado, sem criptografia nem aviso.

Condições de explorabilidade e verificação

Depende do usuário usar o widget Notas para dados sensíveis. Os arquivos ficam no perfil do usuário (permissões herdadas da pasta do perfil).

Impacto

Cópias adicionais de dados sensíveis em disco e em arquivos de exportação compartilháveis.

Correção sugerida

Avisar no widget que o conteúdo não é cifrado; excluir Notes dos snapshots e da exportação por padrão ou proteger com DPAPI (ProtectedData, escopo do usuário).

20. Abuso de requisições (DoS/DDoS)

Severidade	Arquivo:linha	Descrição
BAIXA	Rules.cs:13-14 App.xaml.cs:126-132	**A-03** — Regra de captura usa regex sem limite de tempo: backtracking catastrófico trava a interface *Status: Corrigido*
BAIXA	FenceWindow.cs:377-378 FenceWindow.cs:342	**A-04** — Pré-visualização lê a linha inteira de arquivos de texto na thread da interface (consumo de memória sem limite) *Status: Corrigido*

> BAIXA **A-03** — Regra de captura usa regex sem limite de tempo: backtracking catastrófico trava a interface

Status após a revisão de 18 de outubro de 2026: Corrigido. Regex substituído por casador de curingas iterativo (Rules.cs). O padrão patológico contra nome de 250 caracteres responde em menos de 50 ms (teste automatizado).

Onde (arquivo:linha) e trecho — estado auditado

src/GWindows.Core/Rules.cs:13-14

```
var rx = "^" + Regex.Escape(raw).Replace("\\*", ".*").Replace("\\?", ".") + "$";
if (Regex.IsMatch(name, rx, RegexOptions.IgnoreCase)) return true;
```

src/GWindows.App/App.xaml.cs:126-132

```
void On(string p) => Current.Dispatcher.InvokeAsync(() => { if (Rules.Target(Layout.Fences, p) is { } f) ... });
w.Created += ...; w.Renamed += ...;
```

Por que é explorável

Cada '*' vira '.*'. Uma regra com vários '*' e um nome de arquivo sem o sufixo esperado provoca backtracking exponencial. Medição com a regra *a*a*a*a*a*a*a*a*c e nome 'a'×N + 'b': N=20 → 19 ms; N=24 → 75 ms; N=26 → 163 ms; N=28 → 330 ms; N=30 → 628 ms (dobra a cada +2 caracteres). Nomes de arquivo chegam a 255 caracteres, então o tempo se torna praticamente ilimitado. A avaliação ocorre na thread de interface (Dispatcher.InvokeAsync) a cada arquivo criado ou renomeado no Desktop.

Condições de explorabilidade e verificação

A regra precisa ter vários curingas: possível por digitação própria ou, mais realista, por layout importado (ver A-01). O gatilho é um arquivo novo no Desktop com muitos caracteres iguais ao do padrão (nomes comuns com muitos 'a' bastam).

Impacto

Congelamento da interface (e das cercas) até encerrar o processo.

Correção sugerida

Trocar o regex por um casador de curingas iterativo (O(n·m)) ou construir o Regex com matchTimeout curto (ex.: 50 ms) e tratar RegexMatchTimeoutException como 'não casa'; limitar a quantidade de '*' por padrão.

BAIXA

A-04 — Pré-visualização lê a linha inteira de arquivos de texto na thread da interface (consumo de memória sem limite)

Status após a revisão de 18 de outubro de 2026: Corrigido. Pré-visualização lê no máximo 4 KB e não decodifica imagens acima de 30 MB (FenceWindow.cs, ReadHead).

Onde (arquivo:linha) e trecho — estado auditado

src/GWindows.App/FenceWindow.cs:377-378

```
if (ext is ".txt" or ".md" or ".log" or ".json" or ...)
    return string.Join("\n", File.ReadLines(path).Take(6).Select(l => l.Length > 80 ? l[..80] : l)) + ...;
```

src/GWindows.App/FenceWindow.cs:342

```
tip.Opened += (_, _) => { tip.Content = Preview(path); Raise(tip); };
```

Por que é explorável

File.ReadLines carrega cada linha inteira em memória antes do corte para 80 caracteres. Teste com a mesma expressão: arquivo de 300 MB sem quebra de linha → 1055 ms e pico de 2251 MB de working set (≈7× o tamanho do arquivo, em UTF-16). A chamada ocorre ao abrir o tooltip, na thread de interface.

Condições de explorabilidade e verificação

Basta passar o mouse sobre um arquivo .txt/.md/.log/.json/.csv/.xml/.cs gigante de linha única (p.ex. JSON minificado ou log). A cerca inteligente 'Maiores Arquivos' lista justamente arquivos acima de 100 MB, o que torna o caso realista.

Impacto

Travamento de segundos e pico de memória de gigabytes, podendo derrubar o processo.

Correção sugerida

Abrir com FileStream/StreamReader e ler no máximo alguns KB (ex.: 4 KB) ignorando arquivos acima de um limite; fazer a leitura fora da thread de interface.

Outros (fora da checklist)

Severidade	Arquivo:linha	Descrição
INFORMATIVA	FenceWindow.cs:180	**A-06** — Explorer iniciado sem caminho completo e com argumento montado por concatenação *Status: Corrigido*
INFORMATIVA	App.xaml.cs:89-90	**A-07** — Mutex de instância única com nome previsível permite impedir a abertura do aplicativo *Status: Parcial*
INFORMATIVA	App.xaml.cs:15-24	**A-08** — Início automático aponta para binário não assinado em pasta gravável pelo usuário *Status: Aceito*

INFORMATIVA **A-06** — Explorer iniciado sem caminho completo e com argumento montado por concatenação

Status após a revisão de 18 de outubro de 2026: Corrigido. Explorer iniciado pelo caminho completo da pasta do Windows e com aspas removidas do argumento (FenceWindow.cs).

Onde (arquivo:linha) e trecho — estado auditado

src/GWindows.App/FenceWindow.cs:180

```
Add(L.T("Mostrar no Explorer"), () => Process.Start("explorer.exe", $"/select,\"{path}\""));
```

Por que é explorável

Process.Start com nome relativo e UseShellExecute=false usa a ordem de busca do CreateProcess, que inclui primeiro o diretório do aplicativo (%LocalAppData%\Programs\G-Windows, gravável pelo usuário). Um explorer.exe plantado ali seria executado. O caminho também é interpolado dentro de aspas na linha de comando.

Condições de explorabilidade e verificação

Exige permissão de escrita na pasta do aplicativo (o mesmo usuário), logo não há elevação de privilégio. Não reproduzido dinamicamente: baseado na ordem de busca documentada do Windows. Nomes de arquivo existentes não podem conter aspas, o que limita o risco do argumento.

Impacto

Execução de binário plantado com os privilégios do próprio usuário (persistência/engano).

Correção sugerida

Usar o caminho completo (Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.Windows), "explorer.exe")) e ProcessStartInfo.ArgumentList.

INFORMATIVA **A-07** — Mutex de instância única com nome previsível permite impedir a abertura do aplicativo

Status após a revisão de 18 de outubro de 2026: Parcial. A segunda instância agora exibe um aviso em vez de encerrar em silêncio; o nome do mutex continua previsível (risco aceito).

Onde (arquivo:linha) e trecho — estado auditado

src/GWindows.App/App.xaml.cs:89-90

```
_single = new Mutex(true, "GWindows.SingleInstance", out var first);
if (!first) { Shutdown(); return; }
```

Por que é explorável

Qualquer processo do mesmo usuário que crie antes o mutex 'GWindows.SingleInstance' faz o aplicativo encerrar silenciosamente, sem mensagem.

Condições de explorabilidade e verificação

Exige executar código como o mesmo usuário; sem ganho de privilégio.

Impacto

Negação de serviço local silenciosa na abertura.

Correção sugerida

Incluir identificador do usuário/versão no nome e, ao detectar instância existente, avisar ou ativar a instância atual em vez de encerrar sem aviso.

INFORMATIVA **A-08** — Início automático aponta para binário não assinado em pasta gravável pelo usuário

Status após a revisão de 18 de outubro de 2026: Aceito. Sem alteração: binário não assinado, uso pessoal.

Onde (arquivo:linha) e trecho — estado auditado

src/GWindows.App/App.xaml.cs:15-24

```
const string RunKey = @"Software\Microsoft\Windows\CurrentVersion\Run"; ... k.SetValue("G-Windows",
$"\"{Environment.ProcessPath}\"");
```

Por que é explorável

O valor da chave Run (HKCU) é corretamente entre aspas, mas o executável publicado em %LocalAppData%\Programs\G-Windows não é assinado e a pasta é gravável pelo usuário; quem escrever ali passa a ser executado em cada login.

Condições de explorabilidade e verificação

Mesmo usuário; sem elevação. Prática comum em aplicativos por usuário.

Impacto

Facilita persistência de código no perfil do usuário.

Correção sugerida

Assinar o binário (Authenticode) e, se necessário, instalar em pasta com ACL restrita (ex.: MSIX ou %ProgramFiles%).

Achados por arquivo (linha a linha)

Arquivo	Linha(s)	Achado	Severidade
src/GWindows.App/App.xaml.cs	15-24	**A-08**	INFORMATIVA
src/GWindows.App/App.xaml.cs	89-90	**A-07**	INFORMATIVA
src/GWindows.App/App.xaml.cs	93-95	**A-02**	BAIXA
src/GWindows.App/App.xaml.cs	126-132	**A-03**	BAIXA
src/GWindows.App/App.xaml.cs	191, 218	**A-05**	INFORMATIVA
src/GWindows.App/App.xaml.cs	213	**A-01**	MÉDIA
src/GWindows.App/App.xaml.cs	223	**A-01**	MÉDIA
src/GWindows.App/FenceWindow.cs	180	**A-06**	INFORMATIVA
src/GWindows.App/FenceWindow.cs	255, 351	**A-01**	MÉDIA
src/GWindows.App/FenceWindow.cs	267, 271, 279	**A-02**	BAIXA
src/GWindows.App/FenceWindow.cs	271, 279	**A-01**	MÉDIA
src/GWindows.App/FenceWindow.cs	296-297	**A-05**	INFORMATIVA
src/GWindows.App/FenceWindow.cs	342	**A-04**	BAIXA
src/GWindows.App/FenceWindow.cs	377-378	**A-04**	BAIXA
src/GWindows.App/FenceWindow.cs	395	**A-01**	MÉDIA
src/GWindows.App/Palette.cs	54	**A-01**	MÉDIA
src/GWindows.Core/Layout.cs	24	**A-05**	INFORMATIVA
src/GWindows.Core/Layout.cs	49	**A-01**	MÉDIA
src/GWindows.Core/Layout.cs	49-50	**A-02**	BAIXA
src/GWindows.Core/Rules.cs	13-14	**A-03**	BAIXA
src/GWindows.Core/Rules.cs	21	**A-02**	BAIXA

Arquivos auditados sem achados (todas as linhas lidas): src/GWindows.App/Animator.cs, src/GWindows.App/DarkRenderer.cs, src/GWindows.App/L.cs, src/GWindows.App/Native.cs, src/GWindows.App/Prompt.cs, src/GWindows.App/ShellLink.cs, src/GWindows.App/ShellMenu.cs, src/GWindows.App/TogglePill.cs, src/GWindows.Core/FenceGrid.cs, src/GWindows.Core/Motion.cs, src/GWindows.Core/Smart.cs, src/GWindows.Core/Spring.cs, tests/GWindows.Core.Tests/CoreTests.cs.

Recomendações priorizadas

Prioridade	Ação	Detalhe
P1	**Validar layouts importados/restaurados (A-01, A-02)**	Rejeitar caminhos UNC/dispositivo, normalizar nulos e faixas, limitar tamanhos e pedir confirmação ao importar. Corrige a única falha de severidade média e a de falha na inicialização.
P2	**Substituir o regex da regra de captura (A-03)**	Casador de curingas iterativo ou matchTimeout curto. Evita congelar a interface com uma regra ou arquivo malicioso.
P2	**Limitar a leitura da pré-visualização (A-04)**	Ler poucos KB com StreamReader, recusar arquivos grandes e sair da thread de interface.
P3	**Endurecer a execução local (A-06, A-07, A-08)**	Caminho completo do Explorer + ArgumentList; nome de mutex único por usuário com aviso; assinatura do binário.
P3	**Tratar dados das Notas (A-05)**	Avisar que não são cifradas e excluí-las de snapshots/exportação ou cifrar com DPAPI.
P4	**Manutenção contínua**	Reexecutar dotnet list package --vulnerable a cada release; considerar packages.lock.json para travar versões transitivas.

P1 = corrigir primeiro; P2 = próxima iteração; P3 = endurecimento; P4 = rotina.

ISSUES PARA O GITHUB

Texto pronto para copiar e colar. As issues agrupam achados triviais relacionados (A-06, A-07 e A-08 viram uma só) para evitar excesso de issues. A-01 a A-04 e A-05 têm issue própria.

--- ISSUE 1 ---

```
# [Segurança] Layout importado/restaurado sem validação (caminhos UNC acessados e executados)

**Labels sugeridas:** `security`, `severidade:média`

## Descrição do problema
Importar layout (App.xaml.cs:223) e restaurar snapshot (App.xaml.cs:213) carregam o JSON com LayoutStore.Load e entregam as cercas a Reload sem nenhuma validação. Itens e Portal aceitam qualquer caminho, inclusive \\host\share\arquivo.exe. Ao abrir a cerca o aplicativo faz File.Exists/Directory.Exists, enumera o portal, observa a pasta e extrai o ícone (FenceWindow.cs:271, 279, 308, 395), o que faz o Windows contatar o servidor SMB (tentativa de autenticação NTLM). O item então abre com um clique (FenceWindow.cs:255, 351).

## Evidência
```
src/GWindows.App/App.xaml.cs:223
  Snapshot(); Reload(LayoutStore.Load(d.FileName).Fences);
src/GWindows.Core/Layout.cs:49
  JsonSerializer.Deserialize<LayoutFile>(File.ReadAllText(p))
src/GWindows.App/FenceWindow.cs:279
  return _d.Items.Where(p => File.Exists(p) || Directory.Exists(p));
Teste: Portal=\\attacker.example\share e Items[0]=\\attacker.example\share\x.exe foram aceitos sem alteração.
```

## Impacto
Vazamento de hash NTLM no carregamento, execução remota com 1 clique e atalho falsificado (nome e ícone controlados pelo atacante).

## Sugestão de correção
Validar o layout após desserializar: aceitar apenas caminhos locais totalmente qualificados; rejeitar UNC e prefixos \\?\ e \\.\; limitar tamanho do arquivo e quantidade de itens; exibir confirmação com o resumo do que será importado; não consultar caminhos de rede sem consentimento.

## Critérios de aceite
- [ ] Importar um layout com item ou portal UNC é recusado (ou exige confirmação explícita) e nenhuma conexão SMB é feita ao carregar.
- [ ] Teste automatizado cobre caminhos UNC, \\?\ e relativos no validador.
- [ ] Arquivos maiores que o limite definido são rejeitados com mensagem clara.
- [ ] Restaurar snapshot aplica a mesma validação.
```

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

```
# [Segurança] layout.json sem validação de schema/nulos/faixas derruba o aplicativo na abertura

**Labels sugeridas:** `security`, `severidade:baixa`

## Descrição do problema
LayoutStore.Load aceita JSON válido com campos nulos e valores extremos e só trata JsonException (Layout.cs:49-50). Fences/Items/Name nulos causam NullReferenceException em App.xaml.cs:93-95, FenceWindow.cs:267-279 e Rules.cs:21; o fallback .bak não é usado nesse caso.

## Evidência
```
src/GWindows.Core/Layout.cs:49-50
  try { ... Deserialize<LayoutFile>(File.ReadAllText(p)) ?? new(); } catch (JsonException) { }
Testes executados: {"Fences":null} -> Fences null; {"Items":null,"Name":null} aceitos; Rules.Target com Items=null -> NullReferenceException; W=2147483647, H=-5 aceitos.
```

## Impacto
Queda do aplicativo a cada inicialização enquanto o arquivo adulterado existir; janelas com geometria absurda.

## Sugestão de correção
Normalizar o objeto desserializado (nulos -> padrão, limites de X/Y/W/H, quantidade e tamanho de textos) e capturar qualquer Exception para cair no .bak e depois no layout padrão.

## Critérios de aceite
- [ ] Layout com Fences/Items/Name/Excluded nulos abre o aplicativo com valores padrão, sem exceção.
```

```
- [ ] Geometria fora dos limites é ajustada ao carregar.
- [ ] Falha de leitura (IOException/UnauthorizedAccess) cai no .bak e depois no padrão.
- [ ] Testes unitários cobrem os três casos acima.
--- FIM ISSUE 2 ---
```

```
--- ISSUE 3 ---
# [Segurança] Regra de captura com regex sem timeout (ReDoS) trava a interface

**Labels sugeridas:** `security`, `severidade:baixa`

## Descrição do problema
Rules.Matches converte cada '*' em '.*' e usa Regex.IsMatch sem timeout (Rules.cs:13-14). A chamada ocorre na thread de
interface para cada arquivo novo no Desktop (App.xaml.cs:126-132).

## Evidência
```
src/GWindows.Core/Rules.cs:13-14
  var rx = "^" + Regex.Escape(raw).Replace("*", ".*")...;
Medição (regra *a*a*a*a*a*a*a*a*c, nome 'a'xN+'b'): N=20 19 ms | N=24 75 ms | N=26 163 ms | N=28 330 ms | N=30 628 ms
(dobra a cada +2).
```

## Impacto
Congelamento da interface e das cercas por tempo praticamente ilimitado.

## Sugestão de correção
Casador de curingas iterativo (O(n·m)) ou Regex com matchTimeout curto tratando RegexMatchTimeoutException como 'não
casa'.

## Critérios de aceite
- [ ] Regra *a*a*a*a*a*a*a*a*c contra nome de 255 'a' + 'b' retorna em menos de 50 ms.
- [ ] Teste unitário cobre o caso patológico.
- [ ] Comportamento dos padrões existentes (*.pdf; invoice?) permanece igual (testes atuais passam).
--- FIM ISSUE 3 ---
```

```
--- ISSUE 4 ---
# [Segurança] Pré-visualização lê linha inteira de arquivos grandes na thread de interface

**Labels sugeridas:** `security`, `severidade:baixa`

## Descrição do problema
Preview() usa File.ReadLines(path).Take(6) e só depois corta cada linha para 80 caracteres (FenceWindow.cs:377-378),
chamado ao abrir o tooltip (FenceWindow.cs:342).

## Evidência
```
src/GWindows.App/FenceWindow.cs:378
  string.Join("\n", File.ReadLines(path).Take(6).Select(l => l.Length > 80 ? l[..80] : l))
Teste: arquivo de 300 MB em linha única -> 1055 ms e pico de 2251 MB de working set.
```

## Impacto
Travamento e pico de memória de gigabytes ao passar o mouse sobre um arquivo grande de linha única (p.ex. na cerca
'Maiores Arquivos').

## Sugestão de correção
Ler no máximo ~4 KB com StreamReader, ignorar arquivos acima de um limite e executar fora da thread de interface.

## Critérios de aceite
- [ ] Passar o mouse sobre um .txt de 300 MB sem quebras de linha não aumenta a memória em mais de alguns MB.
- [ ] A interface não bloqueia durante a pré-visualização.
- [ ] Teste automatizado do leitor limitado cobre linha única gigante.
--- FIM ISSUE 4 ---
```

--- ISSUE 5 ---

[Segurança] Notas em claro replicadas em snapshots e exportações

Labels sugeridas: `security`, `severidade:informativa`

Descrição do problema
O texto do widget Notas é gravado em layout.json (FenceWindow.cs:296-297; Layout.cs:24) e copiado para snapshots (App.xaml.cs:191) e exportações (App.xaml.cs:218).

Evidência
```
src/GWindows.App/FenceWindow.cs:297
  box.TextChanged += (_, _) => { _d.Notes = box.Text; App.SaveSoon(); };
src/GWindows.App/App.xaml.cs:191
  File.Copy(LayoutPath, Path.Combine(SnapDir, ...), true);
```

Impacto
Cópias de dados potencialmente sensíveis em disco por até 30 dias e em arquivos exportados.

Sugestão de correção
Avisar o usuário no widget, excluir Notes de snapshots/exportação por padrão ou cifrar com DPAPI (escopo do usuário).

Critérios de aceite
- [ ] O widget informa que o conteúdo não é cifrado.
- [ ] Snapshots e exportações não contêm o texto das notas (ou o contêm cifrado).
- [ ] Teste verifica que a exportação padrão não inclui Notes.

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

[Segurança] Endurecer execução local: explorer.exe sem caminho completo, mutex previsível e binário não assinado

Labels sugeridas: `security`, `severidade:informativa`

Descrição do problema
Três pontos de baixo risco (mesmo usuário, sem elevação): FenceWindow.cs:180 inicia 'explorer.exe' sem caminho completo e com argumento concatenado; App.xaml.cs:89-90 usa mutex de nome previsível que permite impedir a abertura; App.xaml.cs:15-24 registra no Run um executável não assinado em pasta gravável pelo usuário.

Evidência
```
src/GWindows.App/FenceWindow.cs:180
  Process.Start("explorer.exe", $"/select,\"{path}\"")
src/GWindows.App/App.xaml.cs:89-90
  _single = new Mutex(true, "GWindows.SingleInstance", out var first); if (!first) { Shutdown(); return; }
src/GWindows.App/App.xaml.cs:23
  k.SetValue("G-Windows", $"\"{Environment.ProcessPath}\"");
```

Impacto
Execução de binário plantado ou bloqueio silencioso do aplicativo por código já em execução como o mesmo usuário.

Sugestão de correção
Usar caminho completo do Explorer e ArgumentList; nome de mutex com identificador do usuário e aviso ao detectar instância; assinar o executável.

Critérios de aceite
- [ ] 'Mostrar no Explorer' usa caminho absoluto do Windows e ArgumentList.
- [ ] Segunda instância avisa/ativa a primeira em vez de encerrar sem mensagem.
- [ ] O executável publicado é assinado (ou a decisão de não assinar está documentada).

--- FIM ISSUE 6 ---